



INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Department of Electronics, Telecommunications and Computer Engineering

Computer Vision and Mixed Reality

**1st Project – Feature Based Face Detection and Recognition for
Augmented Reality**

Sofia Condesso - 50308

Conducted in the scope of Artificial Vision and Mixed Reality, of the MSc course in Computer and Multimedia Engineering Summer Semester 2022-2023.

April, 2023

Contents

1	Introduction	2
2	Face detection	2
2.1	Haar Cascade Classifier	2
2.2	OpenCV Deep Neural Network module (dnn)	2
2.3	dlib toolbox	3
2.3.1	Real time face detection	3
3	Face Recognition	4
3.1	Face database	4
3.2	Face normalization	4
3.3	Using EigenFaces	4
3.3.1	Training phase (fit)	4
3.3.2	Reconstructions and error faces	5
3.4	Using FisherFaces	6
3.4.1	Training phase (fit)	6
3.4.2	Reconstructions and error faces	6
3.5	Classification - Nearest Neighbour	7
3.5.1	Recognize faces with Eigenfaces	7
3.5.2	Recognize faces with Fisherfaces	8
4	Combine real and virtual objects.	8
4.1	Virtual object normalization	8
4.2	Adding with mask	8
5	Conclusions	9

1 Introduction

This project aims to explore and implement face detection and face recognition algorithms using various techniques such as **Haar Cascade Classifier**, **OpenCV Deep Neural Network** module, and **Dlib toolbox**. For face recognition, **Eigenfaces** and **Fisherfaces** algorithms will be implemented and, then, a **Nearest-Neighbour** classifier will be developed to recognize faces based on both algorithms.

Also, this project has a part where the student is invited to **combine real and virtual objects**, performing **virtual object normalization**.

2 Face detection

2.1 Haar Cascade Classifier

The Haar Cascade Classifier is an algorithm for object detection that uses a set of feature-based cascade classifiers to identify objects in an image. It was proposed by Paul Viola and Michael Jones in their 2001 paper, "Rapid Object Detection using a Boosted Cascade of Simple Features". The algorithm uses a machine learning approach, where a classifier is trained to differentiate between positive (object, e.g., face) and negative (non-face) samples using a set of training images. It is then applied to an input image by sliding a window over it, computing the Haar-like features at each position. If the classifier output exceeds a certain threshold, the window is classified as containing an object.



Figure 1: Face detection with Haar Cascade classifier.

As can be observed by this examples, with several faces, the algorithm fails in one face, one false negative, and several false positives, which may indicate that is not very trustable and better algorithms may exist.

2.2 OpenCV Deep Neural Network module (dnn)

The OpenCV Deep Neural Network (dnn) module is a library of pre-trained convolutional neural networks (CNNs) that can be used for object detection and classification tasks, including face detection. CNNs are a type of deep learning algorithm that are particularly well-suited for image recognition tasks. The dnn module includes a pre-trained ResNet architecture with a Single Shot Detector (SSD) for face detection, which can be used to quickly and accurately identify faces in an input image, but it does not support fine-tuning and training.

Although very promising, it was not possible to try this algorithm, due to errors related to the Python environment and the MacOS version.

2.3 dlib toolbox

Dlib is a toolkit of machine learning algorithms and tools that includes methods for face and facial landmark detection, as well as other computer vision tasks. The face detector uses the Histogram of Oriented Gradients (HOG) feature, combined with a linear classifier, to identify faces in an input image. The HOG feature is computed by dividing the image into small, overlapping regions and computing the gradient orientations and magnitudes in each region. The resulting feature vector is then used to classify each region as containing a face or not.

The dlib library provides two functions that can be used for face detection:

1. `dlib.get_frontal_face_detector()` - Dlib's HOG + Linear SVM face detector is fast and efficient. It is not invariant to changes in rotation and viewing angle, because of the way the Histogram of Oriented Gradients (HOG) descriptor works. The `get_frontal_face_detector()` function does not accept any parameters and returns the pre-trained HOG + Linear SVM face detector, which can be saved to a variable, say "detector" and used like: `detector(image, 1)`, where the "1" in the second argument indicates that we should upsample the image 1 time. This will make everything bigger and allow us to detect more faces. [1]
2. `dlib.cnn_face_detection_model_v1(modelPath)` - For more robust face detection, the MMOD CNN face detector is available through `cnn_face_detection_model_v1` function. This method accepts a single parameter, "modelPath", which is the path to the pre-trained file.

The first method was tried, and outputted better results than Haar Cascade.

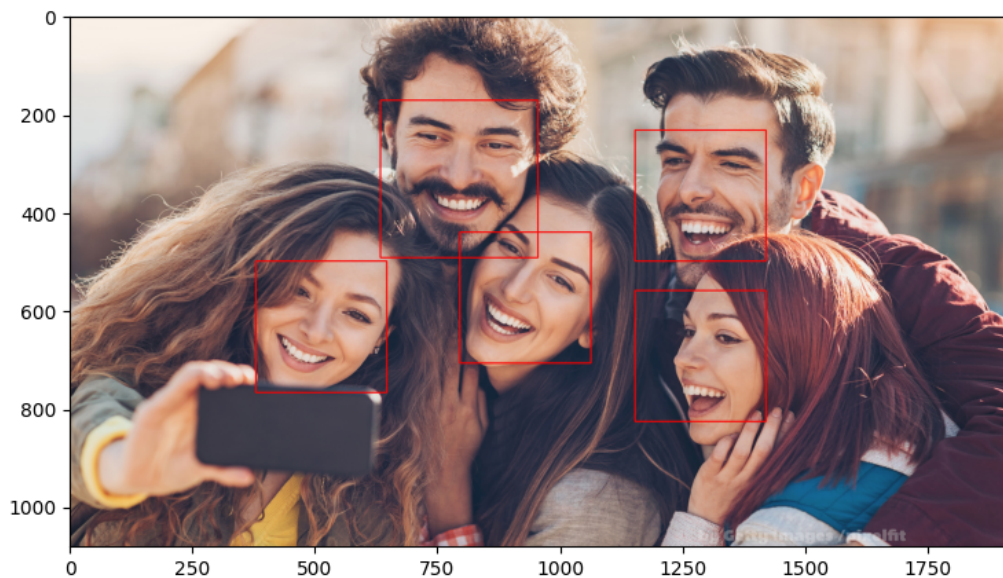


Figure 2: Face detection using HOG + Linear SVM from Dlib.

The results are much better with HOG + Linear SVM than with Haar Cascade classifier, where several false positives were found.

2.3.1 Real time face detection

To perform real time face detection, it was decided to use the best algorithm between the two experimented, the HOG + Linear SVM from Dlib.

The script with the application to try the real time face detection is called `dlib-video.py`.

3 Face Recognition

3.1 Face database

For this project, is used a dataset from Kaggle, a platform with a great number of machine learning resources. The chosen dataset is called "Face Recognition Dataset" [2], and consists in images of 31 celebrities. There is a folder with the original images, which have different resolutions, angles and framing, but there's also a folder with already normalized images (160x160), which we only need to resize to the desired size (56x46), and use in the next steps. This choice is to avoid error propagations, because from the exercise of face normalizing can result errors and, if those images with errors are used in the next steps, they can deteriorate the performance of Eigenfaces and Fisherfaces. However, once implemented correctly, the algorithms can be tested on the normalized data from the exercise.

The train and test sets are produced using `train_test_split()` function from Sklearn, with a test portion of 30%.

3.2 Face normalization

Face normalization is a process of standardizing the size, orientation, and appearance of face images to facilitate their comparison and recognition. In this project, face normalization is performed by rotating, scaling, and selecting a region of interest (ROI) in each face image so that the eyes are perfectly aligned horizontally and located in line 24, columns 16 and 31, respectively. The resulting images are then resized to 56x46 pixels and converted to grayscale with 256 levels.

The images were normalized through these steps:

1. Detect the face in the image and define the face square. Dlib `frontal_face_detector()` was chosen to this task.
2. Use an eye detector on face square and get the eyes' positions.
3. Calculate the angle between eyes.
4. Rotate the eyes from the center, to align horizontally.
5. Resize the image to have 56 rows and 46 columns.

3.3 Using EigenFaces

Eigenfaces is a face recognition algorithm that is based on the use of principal component analysis (PCA) to extract a set of "eigenfaces" from a training set of face images. It was introduced in 1991 by Matthew Turk and Alex Pentland in their paper "Eigenfaces for Recognition". Each eigenface is a vector that represents a specific pattern of variation in the face images, such as changes in lighting or facial expression. The eigenfaces are sorted by their contribution to the overall variance in the training set and used to construct a lower-dimensional subspace that captures the essential features of the face images.

In the Jupyter Notebook code, we have a class called `Eigenfaces` which has the `fit()`, `predict()` and `get_error_faces_and_reconstructions()` methods implemented.

3.3.1 Training phase (fit)

Assuming we have a training set, `Xtrain`, with images of size (56, 46), where `n` equals to 2576 (56*46), `N` equals to the length of `Xtrain`, and `m` equals to 20 (number of eigenvectors), the training phase for the Eigenfaces algorithm proceeds as follows:

1. Apply `np.flatten()` to all images in the dataset, forcing them to be of size (`n`, 1).
2. Compute the mean face, using the function `np.mean()`, and also reshape it to (`n`, 1). The purpose of computing the mean vector is to find the average face in the training set, which will be used as a reference to compare and compute the differences between each individual face in the dataset.
3. Compute matrix `A` (`N`, `n`) by subtracting the mean face from each individual face, highlighting the deviations from the mean that each image has. Matrix `A` is a set of vectors (faces) that are now centered to zero, which is necessary for computing the covariance matrix and the eigenfaces.

4. Compute matrix R (N, N), by multiplying matrix A (N, n) by it's transpose (n, N).
5. Compute the eigenvector/eigenvalues of R (N, N) and create matrix V (N, m) with the eigenvectors, correspondant to the highest m (20) eigenvalues.
6. Compute matrix W (n, m), by multiplying matrix A (N, n) by matrix V (N, m).
7. Projections into the face subspace can be made, multiplying the transposed matrix W (m, n) by the centered images ($n, 1$).

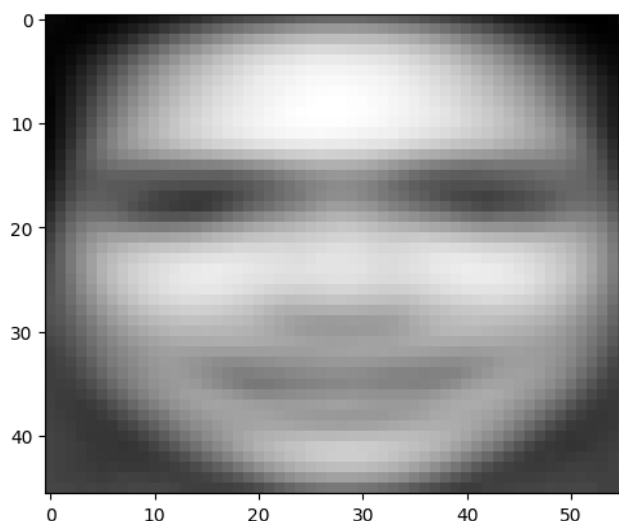


Figure 3: Mean face.

3.3.2 Reconstructions and error faces

To get a view of how a reconstructed and an error face look like, we need to:

1. Perform the projection, on the face subspace, of some training set faces, with the dot product with matrix V .
2. The dot product between the selected projections and the transpose of V , adding the mean-face to that result are the **reconstructed faces**.
3. The difference between the selected faces and the reconstructed faces is the **error face**.

first to perform the projection, on the face subspace, of some training set faces

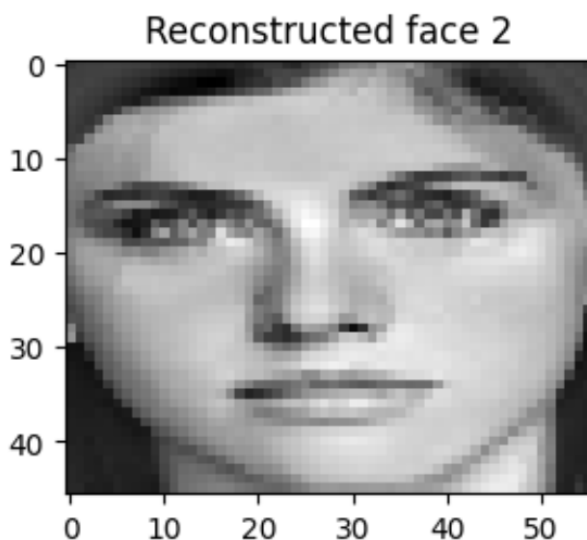


Figure 4: Reconstructed face of an example from train set.

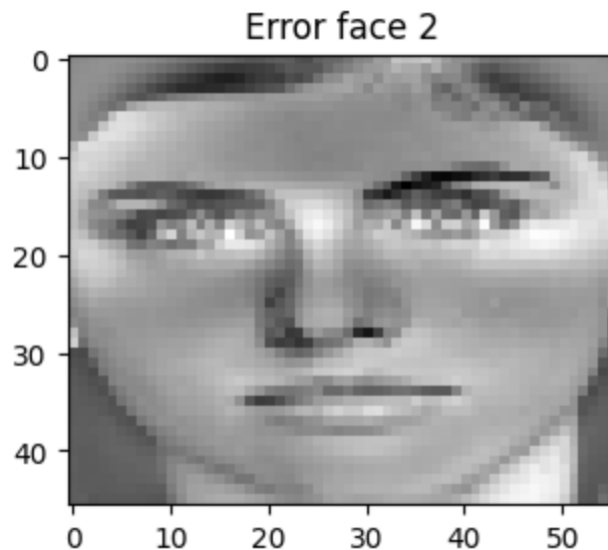


Figure 5: Error face of an example from train set.

3.4 Using FisherFaces

Fisherfaces is a face recognition algorithm that can be seen as an extension of Eigenfaces. Like Eigenfaces, Fisherfaces is a dimensionality reduction technique that aims to represent faces in a lower-dimensional space in which faces of the same person are more similar to each other than faces of different persons. However, unlike Eigenfaces, which uses Principal Component Analysis (PCA) for dimensionality reduction, Fisherfaces uses Multi Discriminant Analysis (MDA) to project the data into a lower-dimensional space.

Similarly to Eigenfaces, in the Jupyter Notebook code, we have a class called `Fisherfaces` which has the `fit()`, `predict()` and `get_error_faces_and_reconstructions()` methods implemented.

3.4.1 Training phase (fit)

The Fisherfaces algorithm goes as follows:

1. Compute the mean face of all dataset.
2. Compute the mean face of each class.
3. Determines the S_b ($n \times n$) and S_w ($n \times n$) matrices, which are measures of the variability between classes and within classes, respectively. These matrices are computed by multiplying W_{pca} by the transpose of the between-class scatter matrix and the within-class scatter matrix.
4. Finally, the algorithm determines the $c-1$ "larger eigenvectors" from the matrix, which are the eigenvectors that capture the most information about the data. These eigenvectors are used to project the data into a lower-dimensional space, in which the Fisherfaces algorithm can be used to classify the data.

3.4.2 Reconstructions and error faces

To get a view of how a reconstructed and an error face look like, in Fisherfaces, we need to:

1. Get selected faces centered, can make use of matrix A .
2. Perform the projection, on the face subspace, of those centered training set faces, with the dot product with matrix W_{fisher} .
3. The dot product between the selected projections and the transpose of W_{fisher} , adding the mean-face to that result are the **reconstructed faces**.
4. The difference between the selected faces and the reconstructed faces is the **error face**.

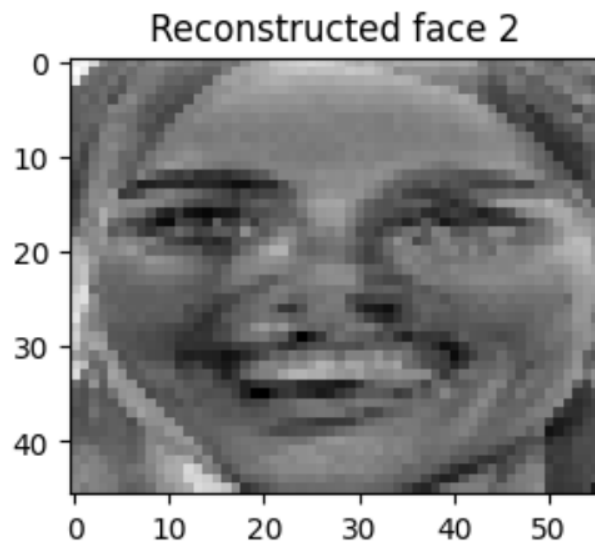


Figure 6: Reconstructed face of an example from train set.

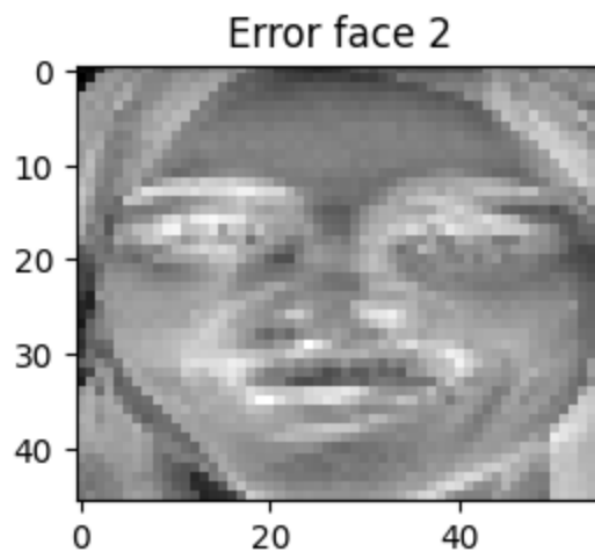


Figure 7: Error face of an example from train set.

3.5 Classification - Nearest Neighbour

To recognize faces with each algorithm, a Nearest Neighbour classifier can be develop, which consists in measuring the distance from the test projection, i.e. the face to recognize, to the feature vectors resulted from the m projection coefficients, and then pick up the face in the face subspace that's more "alike" according to the algorithm.

3.5.1 Recognize faces with Eigenfaces

To recognize a new face, its image is projected onto this subspace and compared to the projections of the training set faces. The closest match is then used to identify the person. In order to do that, we need:

1. Subtract the mean face to the test face, to center the test face.
2. Compute the dot product between the transpose of W and the centered face obtained in step 1.
3. Compute euclidean distances to faces in faces subspace and, the face with less distance is then the recognized face.

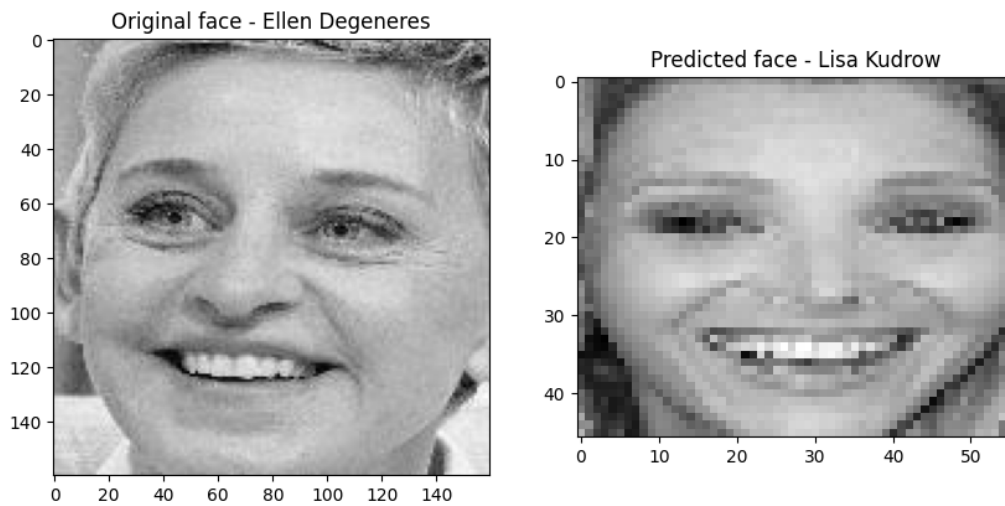


Figure 8: Prediction with Eigenfaces.

3.5.2 Recognize faces with Fisherfaces

To recognize a new face, its image is projected onto this subspace and compared to the projections of the training set faces. The closest match is then used to identify the person.

1. Subtract the mean face to the test face, to center the test face.
2. Compute the dot product between the transpose of W_{fisher} and the centered face obtained in step 1.
3. Compute euclidean distances to faces in faces subspace and, the face with less distance is then the recognized face.

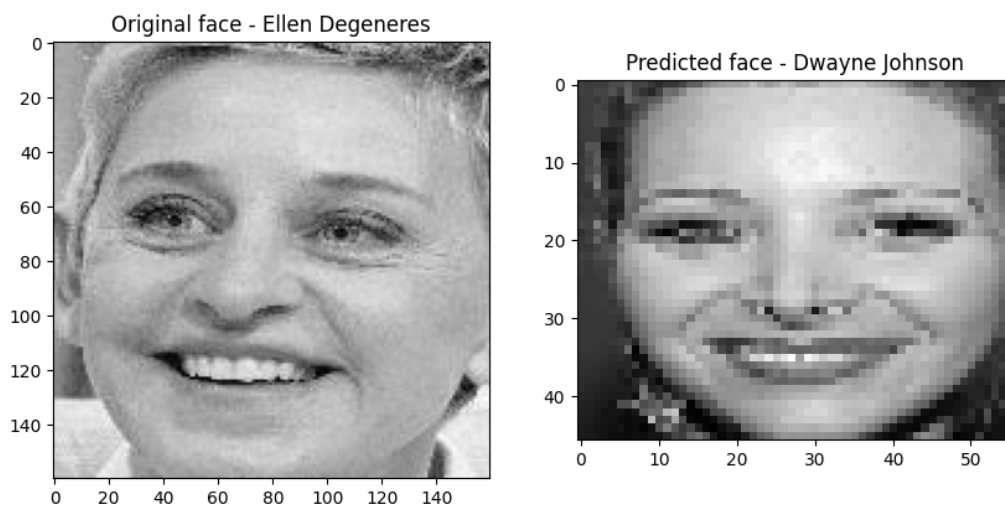


Figure 9: Prediction with Fisherfaces.

4 Combine real and virtual objects.

4.1 Virtual object normalization

Based on the alignment of section b.ii, normalize the virtual object that is intended to be superimposed on the identified face, for example, a hat, glasses, or other object of your choice. Use different objects for different faces/persons.

4.2 Adding with mask

Add to the image, using a mask.

5 Conclusions

In this project, many techniques were experimented, and some, implemented from scratch, which was very challenging and interesting. Different classifiers for face detection were tried and the differences were observed.

In face recognition, Eigenfaces and Fisherfaces showed how linear algebra can have so much importance in this fields, and trying to implement those algorithms was a very rich experience.

References

- [1] Dlib. `get_frontal_face_detector()` source code. http://dlib.net/face_detector.py.html, 2023. Accessed: April 3, 2023.
- [2] Vasuki Patel (in Kaggle). Face recognition dataset. <https://www.kaggle.com/datasets/vasukipatel/face-recognition-dataset>, 2023. Accessed: April 3, 2023.